

1. INFORMACIÓN GENERAL

Table

Campo	Valor
Nombre del Sistema	Fiesta Musical
Versión	2.0
Tipo	Aplicación Web (PWA) + Backend API
Plataforma	PHP-FPM + SQLite + Vanilla JavaScript
Arquitectura	Cliente-Servidor con Long-Polling
Autor	RedTauros
Fecha	2026-06-27

2. DESCRIPCIÓN DEL SISTEMA

Sistema de reproducción musical autónomo donde los oyentes votan por la siguiente canción mientras suena la actual. No requiere operador humano (DJ). La música fluye continuamente mediante transiciones automáticas con efecto fade.

3. USUARIOS DEL SISTEMA

Table

Rol	Descripción	Dispositivo
Oyente/Votante	Usuario que vota por la siguiente canción	Móvil (PWA)
Administrador	Configura géneros, carpetas, y monitorea el sistema	Desktop (Dashboard)

4. REQUERIMIENTOS FUNCIONALES

4.1 Módulo: Gestión de Biblioteca Musical

Table

ID	Requerimiento	Prioridad	Estado
R1.1	El sistema debe permitir configurar múltiples carpetas de música	Alta	✓
R1.2	El sistema debe escanear carpetas y extraer metadatos ID3 (título, artista, álbum, género, duración)	Alta	✓
R1.3	El sistema debe clasificar canciones por género usando: (1) metadatos ID3, (2) nombre de carpeta, (3) género "Desconocido"	Alta	✓
R1.4	El escaneo debe ser incremental (solo archivos nuevos/modificados)	Media	✓
R1.5	El sistema debe soportar formatos: MP3, M4A, FLAC, OGG, WAV, WMA	Alta	✓
R1.6	El sistema debe permitir re-escanear completo manualmente	Media	✓

4.2 Módulo: Gestión de Géneros

Table

ID	Requerimiento	Prioridad	Estado
R2.1	El sistema debe incluir 17 géneros predefinidos: Salsa, Merengue, Bachata, Rock Español, Rock Inglés, Vallenato, Pop, Reggaeton, Balada, Jazz, Electrónica, Cumbia, Champeta, Boleros, Tropical, Cantina, Rancheras	Alta	✓
R2.2	El administrador debe poder activar/desactivar géneros individualmente	Alta	✓
R2.3	El sistema debe detectar automáticamente preferencias del público basado en votos de los últimos 30 minutos	Media	✓
R2.4	Si solo un género está activo, todas las opciones de votación deben ser de ese género	Alta	✓

ID	Requerimiento	Prioridad	Estado
R2.5	Los géneros deben tener icono (emoji) y color asociado para identificación visual	Media	✓

4.3 Módulo: Sistema de Votación

Table

ID	Requerimiento	Prioridad	Estado
R3.1	La votación debe ser continua durante la reproducción de la canción actual (no por rondas)	Alta	✓
R3.2	Un usuario puede votar una sola vez por canción futura, pero puede cambiar su voto en cualquier momento	Alta	✓
R3.3	El sistema debe usar votos con decaimiento temporal : peso = $1 / (1 + \text{horas_desde_voto})$	Alta	✓
R3.4	El sistema debe aplicar bonus de preferencia : +0.02 por cada voto previo del usuario en el mismo género, máximo +0.5	Media	✓
R3.5	El usuario en móvil NO debe ver el ranking de votos (voto ciego)	Alta	✓
R3.6	El usuario solo debe ver su propio voto confirmado	Alta	✓
R3.7	Las 4 opciones de votación deben ser: 2 más votadas históricamente (con decaimiento) + 2 aleatorias de géneros diferentes	Alta	✓

4.4 Módulo: Selección de Canciones

Table

ID	Requerimiento	Prioridad	Estado
R4.1	Al terminar una canción, el sistema debe seleccionar automáticamente la siguiente	Alta	✓
R4.2	Si hay votos, gana la canción con mayor votos ponderados	Alta	✓

ID	Requerimiento	Prioridad	Estado
R4.3	En caso de empate de votos ponderados, desempatar por: canción menos reproducida recientemente	Alta	✓
R4.4	Si no hay votos , seleccionar aleatoriamente ponderado por: menos reproducida	Alta	✓
R4.5	No se requiere mínimo de votos para respetar la elección (1 voto = mayoría)	Alta	✓
R4.6	Implementar anti-repetición configurable : N canciones antes de que una pueda repetirse	Alta	✓
R4.7	N debe ser configurable en el dashboard (default: 10, rango: 0 hasta tamaño_biblioteca - 4)	Alta	✓

4.5 Módulo: Reproducción y Transiciones

Table

ID	Requerimiento	Prioridad	Estado
R5.1	Transición automática entre canciones sin intervención humana	Alta	✓
R5.2	Efecto fade out configurable (default: 2 segundos)	Alta	✓
R5.3	Efecto fade in configurable (default: 1.5 segundos)	Alta	✓
R5.4	Silencio entre canciones configurable (default: 0.5 segundos)	Media	✓
R5.5	La votación para la siguiente canción debe iniciar automáticamente antes de que termine la actual	Alta	✓
R5.6	El sistema debe funcionar en modo fiesta continua (auto-play) por defecto	Alta	✓

4.6 Módulo: Dashboard Administrativo

Table

ID	Requerimiento	Prioridad	Estado
R6.1	Visualizar canción actual con progreso en tiempo real	Alta	✓

ID	Requerimiento	Prioridad	Estado
R6.2	Visualizar resultados de votación en tiempo real (con votos ponderados)	Alta	✓
R6.3	Configurar carpetas de música a escanear	Alta	✓
R6.4	Activar/desactivar géneros	Alta	✓
R6.5	Configurar N de anti-repetición	Alta	✓
R6.6	Configurar duraciones de fade out, fade in, silencio	Media	✓
R6.7	Visualizar logs de actividad del sistema	Media	✓
R6.8	Generar QR code / URL para que móviles se conecten	Media	✓
R6.9	Mostrar estadísticas: total canciones, géneros activos, duración total	Media	✓
R6.10	Mostrar preferencias detectadas automáticamente	Media	✓
R6.11	Controles manuales de emergencia: forzar siguiente, pausar, reanudar	Baja	✓

4.7 Módulo: Interfaz Móvil (PWA)

Table

ID	Requerimiento	Prioridad	Estado
R7.1	Ver canción actual con progreso	Alta	✓
R7.2	Ver 4 opciones para votar (sin números de votos)	Alta	✓
R7.3	Botón para votar por una opción	Alta	✓
R7.4	Indicador visual de "voto registrado"	Alta	✓
R7.5	Poder cambiar voto seleccionado	Alta	✓
R7.6	Funcionar como PWA instalable (offline parcial)	Media	✓

ID	Requerimiento	Prioridad	Estado
R7.7	Diseño responsive para móviles	Alta	✓

4.8 Módulo: Historial

Table

ID	Requerimiento	Prioridad	Estado
R8.1	Registrar historial de canciones reproducidas	Media	✓
R8.2	Incluir votos recibidos por cada canción	Media	✓
R8.3	Mostrar historial en dashboard	Media	✓

5. REQUERIMIENTOS NO FUNCIONALES

Table

ID	Requerimiento	Prioridad
RNF1	Funcionar en red local sin internet	Alta
RNF2	Tiempo de respuesta de API < 500ms	Alta
RNF3	Soporte para 50+ usuarios concurrentes votando	Media
RNF4	Base de datos SQLite embebida (sin servidor de DB externo)	Alta
RNF5	Código PHP compatible con PHP 8.1+	Alta
RNF6	Frontend en Vanilla JavaScript (sin frameworks)	Alta
RNF7	Long-polling para actualizaciones en tiempo real	Alta
RNF8	Diseño dark mode moderno	Media
RNF9	Transiciones CSS animadas	Media
RNF10	Compatibilidad con Chrome, Firefox, Safari, Edge	Alta

6. REGLAS DE NEGOCIO

Table

ID Regla

RN1 Un usuario = un voto activo por vez (identificado por hash de IP + User-Agent)

RN2 Voto cambiable: al votar por nueva canción, se borra el voto anterior

RN3 Decaimiento temporal: voto de hace 5 min = 92% peso, hace 30 min = 67%, hace 2 horas = 33%

RN4 Bonus de preferencia: usuario que vota mucho salsa tiene +peso en salsa

RN5 Anti-repetición: canción no puede salir hasta que pasen N canciones desde su última reproducción

RN6 Si 0 votos: selección aleatoria ponderada por menos reproducida

RN7 Empate: desempatar por canción con reproducción más antigua

RN8 Modo fiesta continua: sistema autónomo, sin operador

7. FLUJOS PRINCIPALES

7.1 Inicio del Sistema

plain

1. Admin abre dashboard
2. Sistema escanea carpetas configuradas (incremental)
3. Admin selecciona géneros activos
4. Sistema inicia modo fiesta continua
5. Reproduce primera canción (aleatoria de géneros activos)
6. Usuarios móviles se conectan y empiezan a votar

7.2 Ciclo de Reproducción

plain

1. Canción A suena
 2. Usuarios votan para la siguiente (voto ciego)
 3. Canción A termina (con fade out)
 4. Sistema determina ganadora por votos ponderados
 5. Fade in de Canción B
 6. Votos se resetean para nueva ronda
 7. Usuarios votan para Canción C
 8. REPETIR
-

8. DICCIONARIO DE DATOS

8.1 Entidades Principales

Table

Entidad	Descripción	Campos Clave
Canción	Archivo musical indexado	id, file_path, title, artist, genre_id, duration, play_count, vote_score
Género	Categoría musical	id, name, color, icon, is_active
Voto	Selección de usuario	id, song_id, user_hash, weight, timestamp
Sesión	Estado actual del sistema	id, status, current_song_id, voting_options, config
Historial	Registro de reproducción	id, song_id, played_at, votes_received
Carpeta	Ruta de música escaneada	id, path, is_active, last_scan

9. INTERFACES

9.1 API REST Endpoints

Table

Endpoint	Método	Descripción
/api/scan	POST	Escanear carpeta de música
/api/genres	GET/POST	Listar/configurar géneros
/api/songs	GET	Listar canciones
/api/config	GET/POST/DELETE	Configuración de sesión
/api/vote	POST	Emitir voto
/api/results	GET	Resultados de votación
/api/status	GET	Long-polling de estado
/api/start-vote	POST	Iniciar votación
/api/finalize-vote	POST	Finalizar votación
/api/play-winner	POST	Reproducir ganadora
/api/history	GET	Historial de reproducción
/api/next-song	GET	Verificar transición automática

10. CRONOGRAMA SUGERIDO

Table

Fase	Duración	Entregables
Fase 1: Backend Core	2-3 días	API completa, base de datos, escaneo
Fase 2: Frontend Móvil	2 días	PWA votación, long-polling
Fase 3: Dashboard	2 días	Panel admin, reproductor, config
Fase 4: Integración	1-2 días	Pruebas, ajustes, deploy
Total	7-9 días	MVP funcional

11. GLOSARIO

Table

Término	Definición
Voto ciego	Usuario no ve cantidad de votos de otros
Decaimiento	Pérdida de peso de voto con el tiempo
Fade	Transición gradual de volumen
Long-polling	Técnica HTTP para simular tiempo real
PWA	Progressive Web App, instalable en móvil
Modo fiesta continua	Sistema autónomo sin intervención
